

SameerBot — Full Project Journey

Step-by-step build log including failures, fixes & final features

Project	WhatsApp AI Bot + AI Platform
Developer	Sameer Shaik
AI Name	Nuh 1.0
Company	N.S (Nadeem Shaik)
Stack	Node.js + Spring Boot + Python + Docker + Ollama
Search	Tavily API (live internet)
Document	Complete build journal with failures & fixes

Table of Contents

- Phase 1 — Project Setup & Initial WhatsApp Bot
- Phase 2 — Spring Boot AI Gateway (First Attempt)
- Phase 3 — Connecting Bot to Spring (404 Fix)
- Phase 4 — application.properties Missing (500 Fix)
- Phase 5 — Docker Setup (Multiple Failures)
- Phase 6 — Ollama Host Networking Fix
- Phase 7 — bot.js Cleanup & Human Mode Feature
- Phase 8 — Nuh Custom Model & Identity
- Phase 9 — Tavily Live Internet Search
- Phase 10 — Open WebUI Integration
- Phase 11 — Python FastAPI Block Service
- Phase 12 — Final Architecture & Commands

Phase 1 — Project Setup & Initial WhatsApp Bot

Goal: Create a WhatsApp bot using Baileys (Node.js) that responds to messages with Ollama AI.

Initial folder structure created:

```
mkdir -p ~/Desktop/whatsapp-bot cd ~/Desktop/whatsapp-bot npm init -y npm install  
@whiskeysockets/baileys axios pino qrcode-terminal
```

bot.js CONFIG:

```
const CONFIG = { authDir: './auth', ollamaUrl: 'http://127.0.0.1:11434/api/generate',  
aiModel: 'qwen2.5:1.5b', };
```

Result: Bot connected to WhatsApp, called Ollama directly. Basic commands worked.

Problem: Architecture was messy — bot talked directly to Ollama, no separation of concerns.

Phase 2 — Spring Boot AI Gateway (First Attempt)

Goal: Add Spring Boot as a middleware between bot and Ollama for better architecture.

Spring Boot project created at:

```
~/Desktop/sameer-ai-platform/services/ai-gateway
```

pom.xml dependencies:

```
org.springframework.boot spring-boot-starter-webmvc org.springframework.ai  
spring-ai-starter-model-ollama
```

FAILURE: ChatController.java placed in wrong folder

```
controller/ChatController.java (WRONG – outside src/) Should be:  
src/main/java/com/sameer/ai_gateway/controller/ChatController.java
```

FIX: Moved file to correct package path inside src/main/java/

Phase 3 — Connecting Bot to Spring (404 Fix)

Goal: Make bot.js call Spring instead of Ollama directly.

Changed bot.js CONFIG:

```
// OLD ollamaUrl: 'http://127.0.0.1:11434/api/generate', // NEW springApiUrl:  
'http://localhost:8080/api/ai/chat',
```

FAILURE 1: curl returned 404

```
curl "http://localhost:8080/api/ai/chat?message=hello" {"status":404,"error":"Not Found"}
```

Cause: ChatController was still in wrong folder — Spring never loaded it.

FIX: Moved controller to src/main/java/com/sameer/ai_gateway/controller/

FAILURE 2: curl returned 500 after fix

```
{"status":500,"error":"Internal Server Error"}
```

Cause: application.properties was missing — Spring had no Ollama config.

Phase 4 — application.properties Missing (500 Fix)

FAILURE: No application.properties file existed

```
ls src/main/resources/application.properties cat: No such file or directory
```

FIX: Created application.properties with Ollama config:

```
spring.application.name=ai-gateway server.port=8080
spring.ai.ollama.base-url=http://localhost:11434 spring.ai.ollama.chat.model=qwen2.5:1.5b
spring.ai.ollama.chat.options.temperature=0.7
```

Test result after fix:

```
curl "http://localhost:8080/api/ai/chat?message=hello" Hello! How can I assist you today?
```

SUCCESS: Spring + Ollama pipeline working!

Phase 5 — Docker Setup (Multiple Failures)

Goal: Containerize everything — one command to start all services.

Files created:

```
services/ai-gateway/Dockerfile services/whatsapp-bot/Dockerfile docker-compose.yml
```

FAILURE 1: docker-compose.yml created as a DIRECTORY

```
cat: docker-compose.yml: Is a directory docker compose up: read docker-compose.yml: is a directory
```

FIX: `rm -rf docker-compose.yml`, then recreate as file

FAILURE 2: Maven SNAPSHOT build failed inside Docker

```
ERROR: Could not transfer artifact from spring-snapshots repo.spring.io: Unknown host
```

Cause: Docker build container had no access to Spring snapshot repo.

FIX: Build jar locally first, then copy into Docker image:

```
# Build on host machine mvn package -DskipTests -q # Dockerfile just copies pre-built jar
FROM eclipse-temurin:21-jre-jammy COPY target/*.jar app.jar ENTRYPOINT ["java", "-jar",
"app.jar"]
```

FAILURE 3: Port 8080 already in use

```
Error: failed to bind host port 0.0.0.0:8080: address already in use
```

FIX: Kill old Spring process before starting Docker:

```
pkill -f ai-gateway
```

FAILURE 4: bot.js had syntax error — await outside async

```
SyntaxError: await is only valid in async functions at bot.js:63
```

Cause: Time handler code was pasted at top level of file, outside any function.

FIX: Removed misplaced code block with `sed -i '54,66d' bot.js`

Phase 6 — Ollama Host Networking Fix

Goal: Make Spring inside Docker reach Ollama running on the host machine.

FAILURE: Connection refused to host.docker.internal:11434

```
finishConnect failed with error(-111): Connection refused
host.docker.internal/172.17.0.1:11434
```

Diagnosis: Ollama only listening on 127.0.0.1 (localhost), not 0.0.0.0

```
ss -tlnp | grep 11434 LISTEN 0 4096 127.0.0.1:11434 0.0.0.0:*
```

FIX: Configure Ollama to listen on all interfaces:

```
sudo mkdir -p /etc/systemd/system/ollama.service.d sudo tee
/etc/systemd/system/ollama.service.d/override.conf << 'EOF' [Service]
Environment="OLLAMA_HOST=0.0.0.0" EOF sudo systemctl daemon-reload sudo systemctl restart
ollama # Verify – must show *:11434 ss -tlnp | grep 11434 LISTEN 0 4096 *:11434 *:*
```

docker-compose.yml key settings for host networking:

```
environment: SPRING_AI_OLLAMA_BASE_URL: http://host.docker.internal:11434 extra_hosts: -
"host.docker.internal:host-gateway"
```

SUCCESS: All containers connected, WhatsApp bot working end-to-end!

Phase 7 — bot.js Cleanup & Human Mode Feature

Goal: Clean up bot.js and add human mode — AI goes silent when user requests Sameer.

Human mode logic added:

```
// Triggers human mode wantsHumanChat('I want to chat with Sameer') -> true  
wantsHumanChat('talk to your boss') -> true // Wakes AI back up isWakingAI('Hey Nuh, what is  
Java') -> true isWakingAI('Nuh explain arrays') -> true
```

Message flow:

User Message	Bot Action
hi / hello	Greeting + show Sameer option
I want to chat with Sameer	Human mode ON — AI silent
talk to your boss	Human mode ON — AI silent
Hey Nuh, <question>	AI wakes up + answers
Any other text	Nuh AI responds normally
!help	Show commands list
!time	Current date and time
!info	Bot identity info
!ask <question>	Force AI answer

Phase 8 — Nuh Custom Model & Identity

Goal: Create custom Ollama model 'Nuh 1.0' with correct identity (Sameer Shaik, N.S).

FAILURE: AI said 'I am Nuh, created by Nadeem Shaik'

Cause: Spring system prompt had wrong creator name.

Created Nuh Modelfile:

```
FROM qwen2.5:1.5b SYSTEM "" You are Nuh, an AI assistant. Version: Nuh 1.0 Created by:
Sameer Shaik Company: N.S Keep replies short. Plain text only. Use - for lists. "" PARAMETER
temperature 0.7 ollama create Nuh -f /tmp/Modelfile
```

Updated Spring system prompt:

```
.defaultSystem("You are Nuh 1.0, an AI assistant created by Sameer Shaik, company N.S. Keep
replies short. Plain text only.")
```

Test result:

```
curl "http://localhost:8080/api/ai/chat?message=who+are+you" I am Nuh, an AI assistant
created by Sameer Shaik.
```

SUCCESS: Identity correct!

Phase 9 — Tavily Live Internet Search

Goal: Give Nuh access to live internet data — news, sports, weather, prices.

FAILURE 1: Used Spring AI @Tool annotation — model too small to use tools

qwen2.5:1.5b does not support function/tool calling.

FAILURE 2: Injected Tavily results into prompt — model ignored them

qwen2.5:1.5b too small to follow injected context instructions.

FAILURE 3: Added Jackson dependency incorrectly to pom.xml

```
ERROR: Non-parseable POM – duplicate dependency tag ERROR: version missing for
jackson-databind
```

FIX: Rewrote pom.xml from scratch, removed Jackson (already in Spring Boot parent)

FINAL FIX: Return Tavily answer directly — bypass Ollama for searches

```
@GetMapping("/chat") public String chat(@RequestParam String message) { if
(!tavilyApiKey.isEmpty() && needsSearch(message)) { String result = searchTavily(message); if
(result != null) return result; // direct Tavily answer } return
chatClient.prompt().user(message).call().content(); }
```

Keywords that trigger search:

```
news, today, latest, current, weather, price, score, 2026, live, who won, stock, cricket,
ipl, match, election, rupee, sensex
```

Test result:

```
curl "http://localhost:8080/api/ai/chat?message=ipl+cricket+score+today" Today IPL score
shows RCB at 161/5, 18 overs completed. 2026 season.
```

SUCCESS: Live internet search working!

Phase 10 — Open WebUI Integration

Goal: Add browser-based chat UI connected to Nuh model.

FAILURE: Open WebUI container was unhealthy

```
Container open-webui: Up 2 minutes (unhealthy) Cause: healthcheck used wget which wasn't in the image
```

FIX: Removed healthcheck dependency, used simple depends_on

Added to docker-compose.yml:

```
open-webui: image: ghcr.io/open-webui/open-webui:main ports: - "3000:8080" environment:
  OLLAMA_BASE_URL: http://host.docker.internal:11434 volumes: -
  open-webui-data:/app/backend/data
```

Access at: <http://localhost:3000>

Features: Select Nuh model, enable DuckDuckGo web search, chat in browser.

SUCCESS: All 3 services (ai-gateway, whatsapp-bot, open-webui) in one docker-compose!

Phase 11 — Python FastAPI Block Service

Goal: Add Python microservice to handle mute/archive blocking. Bot checks Python before responding to any message.

Architecture:

```
WhatsApp message -> Node.js bot -> GET http://python-service:8000/should-respond?jid=... ->
Python returns {respond: true/false} -> If false: bot stays silent -> If true: route to
Spring/AI
```

Python service endpoints (main.py):

Endpoint	Method	Action
/should-respond?jid=...	GET	Returns {respond: true/false}
/mute	POST	Add JID to muted set
/mute	DELETE	Remove JID from muted
/archive	POST	Add JID to archived set
/archive	DELETE	Remove JID from archived
/blocked	GET	List all muted + archived
/health	GET	Health check

Python service files:

```
services/python-service/ main.py # FastAPI app requirements.txt # fastapi, uvicorn, pydantic
Dockerfile # FROM python:3.12-slim blocked.json # persists muted/archived across restarts
```

FAILURE: Duplicate isBlocked functions in bot.js

```
Line 21: function isBlocked(jid) { return mutedUsers.has... } // OLD sync Line 72: async
function isBlocked(jid) { axios.get... } // NEW async Line 237: if (isBlocked(jid)) //
missing await
```

FIX: Rewrote bot.js completely — removed all old Sets, single async isBlocked only

Test results:

```
curl -X POST "http://localhost:8000/mute" -d '{"jid":"919...@s.whatsapp.net"}'
{"status":"muted","jid":"919...@s.whatsapp.net"} curl
"http://localhost:8000/should-respond?jid=919...@s.whatsapp.net"
{"respond":false,"reason":"muted"} curl "http://localhost:8000/blocked"
{"muted":["919..."],"archived":[],"total":1}
```

SUCCESS: Python block service fully working!

Phase 12 — Final Architecture & Commands

Final docker-compose services:

Container	Image	Port	Purpose
ai-gateway	Spring Boot (custom)	8080	AI routing + Tavily search
whatsapp-bot	Node.js (custom)	-	WhatsApp connection + logic
python-service	Python FastAPI (custom)	8000	Mute/archive block service
open-webui	ghcr.io/open-webui	3000	Browser chat UI

Daily commands:

```
# Start everything cd ~/Desktop/sameer-ai-platform && sudo docker compose up -d # Stop everything sudo docker compose down # View bot logs sudo docker compose logs -f whatsapp-bot # Rebuild after code change sudo docker compose up -d --build
```

Mute/Archive commands (from terminal):

```
# Mute a user curl -X POST http://localhost:8000/mute \ -H 'Content-Type: application/json' \ -d '{"jid":"91XXXXXXXXXX@s.whatsapp.net"}' # Unmute curl -X DELETE http://localhost:8000/mute \ -H 'Content-Type: application/json' \ -d '{"jid":"91XXXXXXXXXX@s.whatsapp.net"}' # Archive curl -X POST http://localhost:8000/archive \ -H 'Content-Type: application/json' \ -d '{"jid":"91XXXXXXXXXX@s.whatsapp.net"}' # View all blocked curl http://localhost:8000/blocked
```

WhatsApp bot commands:

Command	Description
hi / hello	Greeting with options
!help	Show all commands
!info	Nuh 1.0 identity info
!ask <question>	Force AI to answer
!time	Current date and time
!ping	Bot response speed
!mute <jid>	Mute user from WhatsApp
!archive <jid>	Archive user from WhatsApp
!unmute <jid>	Unmute user
!unarchive <jid>	Unarchive user
!blocked	List all blocked users
I want to chat with Sameer	Human mode ON
Hey Nuh, <question>	Wake AI from human mode